Callbacks & Webhooks — Whitelist Coverage & Processing Gaps

InstantMudra Enterprises Admin Portal

Callback endpoint inventory — AESFilter whitelist status (String.startsWith() semantics, per Module 4)

Controller	Endpoint	Whitelist status
PGCallbackController	`/api/pg/callback/runpaisa`	✓ covered — exact match
PGCallbackController	`/api/pg/callback/unisus`	✓ covered — exact match
PGCallbackController	`/api/pg/callback/nixasoft`	✓ covered — exact match
PGCallbackController	`/api/pg/callback/slpe`	✓ covered — exact match
PGCallbackController	`/api/pg/callback/razorpay`	✓ covered — exact match
PGCallbackController	`/api/pg/callback/cashfree`	✓ covered — exact match
PGCallbackController	`/api/pg/callback/im`	✓ covered — exact match
PayoutCallbackController	`/api/payout/callback/runpaisa`	✓ covered — exact + prefix
PayoutCallbackController	`/api/payout/callback/nixasoft`	✓ covered — via bare "/api/payout/callback" prefix
PayoutCallbackController	`/api/payout/callback/slpe`	✓ covered — via bare prefix
PayoutCallbackController	`/api/payout/callback/vimopay`	✓ covered — exact + prefix
PayoutCallbackController	`/api/payout/callback/im`	✓ covered — exact + prefix
PayoutCallbackController	`/api/payout/unisus-callback`	✗ **NOT covered — different path shape, no prefix matches**
RechargeCallbackController	`/api/recharge/callback`	✗ **NOT covered — zero whitelist entries for /api/recharge/***

Prefix-matching nuance (built on the Module 4 correction)

The bare whitelist entry "/api/payout/callback" alone covers every "/api/payout/callback/*" sub-path under AESFilter's String.startsWith() matching — the specific per-vendor entries next to it are redundant. But **"/api/payout/unisus-callback"** has a different path shape (no "/callback" segment), so no entry — specific or prefix — covers it.

Signature verification — inconsistent, and disabled where present

Cashfree — check exists, is commented out

boolean isValid = verifySignature(...); // commented out
log.info("SIGNATURE VERIFIED"); // logged regardless, unconditionally, whether or not it actually was

(top-level finding — confirmed directly in source)

Razorpay — signature captured, never checked

razorpay_signature is accepted as a request param and passed to RazorPayPGServiceImpl.processCallback(), but no verification logic references it anywhere in that file — a dead parameter, not a disabled check.

RunPaisa (the default/most-used PG vendor) — zero signature/HMAC/checksum verification anywhere in RunpaisaPGServiceImpl

CallbackParserFactory — a RunPaisa-shaped abstraction, RunPaisa-only in practice

VendorCallbackParser.parse(payload) returns **`RunPaisaCallback`** specifically — the interface itself is shaped around RunPaisa's response, even though CashfreeCallbackParser and RazorpayCallbackParser also implement it.

Confirmed by direct search: CallbackParserFactory.getParser() is called from exactly one place — RunpaisaPGServiceImpl. Cashfree and Razorpay's actual webhook handlers (processWebhook() / processCallback()) never call the factory at all — their CallbackParser implementations are registered Spring beans that nothing ever selects.

Duplicate delivery & failure handling

Idempotency: 1 of ~11 callback endpoints

IdempotencyServiceUtil (real, DB-unique-constraint-backed) protects only the Cashfree PG callback.

Its own key header is optional (required = false) — a vendor retry that omits it bypasses the guard even on this one protected path.

Async failures are silently swallowed

UnisusCallbackAsyncProcessor.processAsync() runs @Async, after the controller already returned 200 OK.

Its entire body is wrapped in try/catch with an explicit "// DO NOT THROW" — any downstream lookup failure leaves the transaction stuck, silently, with no retry.

Whitelisted / covered · Not covered / disabled check — confirmed gap · Flagged risk · Dead / unreachable code